

Movie Recommendation Algorithms

Vladimir Ceban, CSEN 169 - Web Information Management, Winter 2025.

In this assignment, I developed a set of collaborative filtering algorithms to predict movie ratings based on user preferences. The goal was to analyze and compare different recommendation techniques to evaluate their effectiveness.

Usage

This is a JavaScript project that can be run using Node.js. To execute the algorithms, follow these steps:

Make sure you have Node.js version 18 or higher installed on your system.

```
bash node -v
```

Clone the repository or unzip the project files and navigate to the project directory.

```
bash git clone https://github.com/v3ceban/recomendations cd
recomendations
```

or

```
bash unzip recomendations.zip cd recomendations
```

Run `main.js` with Node.js or as a script

```
bash node main.js <test_file> <algorithm>
```

or

```
bash ./main.js <test_file> <algorithm>
```

Where test file is a path to the test file (i.e. `test/test5.txt`) and algorithm is one of the following:

- `user-cosine`
- `user-pearson`
- `user-iuf`
- `user-case`
- `item`
- `hybrid`
- `svd`
- `hybrid-svd`

To redirect program output to a file, use the following command:

```
bash node main.js <test_file> <algorithm> > output.txt
```

Examples

To run the **hybrid** algorithm on the **test5.txt** dataset, use the following command:

```
bash ./main.js test/test5.txt hybrid
```

To run the **svd** algorithm on the **test10.txt** dataset and save the output to a file, use:

```
bash node main.js test/test10.txt svd > output.txt
```

Testing

In addition to main prediction features, the project includes a number of testing scripts that allow to quickly evaluate the performance of each algorithm on the test datasets.

Split Training Data

Divide a training dataset into separate validation and testing sets for more accurate performance measurement:

```
bash npm run split <training_file> [base-name]
```

Where **training_file** is a path to the training dataset (i.e. **train.txt**) and **base-name** is an optional base name for the output files (defaults to test file name).

Evaluate Algorithm

Test the performance of a specific algorithm against a test dataset:

```
bash npm run evaluate <algorithm> <test_file> [validation_file]
```

Where **algorithm** is one of the supported algorithms (**user-cosine**, **user-pearson**, **user-iuf**, **user-case**, **item**, **hybrid**, **svd**, or **hybrid-svd**), **test_file** is the path to the test dataset, and **validation_file** is an optional validation dataset (if not provided, one will be generated automatically from the test file).

Example Evaluates the hybrid algorithm on test5.txt:

```
bash npm run evaluate hybrid test/test5.txt
```

Evaluates the hybrid algorithm on a pre-split test5 file using both test and validation dataset:

```
bash npm run evaluate hybrid test5_test.txt test5_validation.txt
```

Compare Algorithms

Run a benchmark comparing all implemented algorithms on the same dataset:

```
bash npm run compare <test_file> [validation_file]
```

Where `test_file` is the path to the test dataset and `validation_file` is an optional validation file. The output is a table ranking all algorithms by their MAE scores, providing a clear overview of which algorithms perform best on the given dataset.

Example Compares all algorithms on `test10.txt`:

```
bash npm run compare test/test10.txt
```

Compares all algorithms on a pre-split `test10` file using both test and validation dataset:

```
bash npm run compare test10_test.txt test10_validation.txt
```

Data Description

The **training dataset** consists of movie ratings from 200 users on 1000 movies, where each rating is represented as a triple (**User ID**: int, **Movie ID**: int, **Rating**: 1 | 2 | 3 | 4 | 5).

There are three **test datasets**: `test5.txt`, `test10.txt`, and `test20.txt`, each containing 100 users and 5, 10, or 20 movie ratings per user, respectively.

Implemented Algorithms

In total, I developed eight collaborative filtering algorithms that can be divided into three main categories: **user-based**, **item-based**, **hybrid/ensemble**.

User-Based Collaborative Filtering

I implemented four **user-based** algorithms: **user-cosine**, **user-pearson**, **user-iuf**, and **user-case**.

User-Cosine This is a **user-based** recommendation algorithm that measures the similarity between two users by calculating the cosine of the angle between their rating vectors in a multi-dimensional space.

It first identifies common items (movies) that both users have rated. For each common item, the algorithm multiplies the ratings together to calculate the dot product, while also tracking the squared sum of ratings for each user. These values are used to compute the vector magnitudes.

The similarity score is calculated by dividing the dot product by the product of the two vector magnitudes. This produces a value ranging from -1 to 1,

where 1 indicates perfect similarity, 0 indicates no similarity, and negative values would indicate dissimilarity (though in rating contexts, values are typically non-negative).

Cosine similarity works well for sparse data and focuses on the direction of rating patterns rather than their magnitude, making it particularly useful for recommendation systems where users may rate different numbers of items on different scales.

Performance

- **MAE of GIVEN 5:** 0.829561085407028
- **MAE of GIVEN 10:** 0.779
- **MAE of GIVEN 20:** 0.76598823188965
- **OVERALL MAE:** 0.790059103595469

User-Pearson The Pearson correlation is the second **user-based** algorithm I worked on. It measures similarity between users by evaluating the linear relationship between their rating patterns, accounting for individual rating tendencies.

It begins by identifying common items both users have rated and calculates each user's average rating across all their items. For each common item, the algorithm computes the deviation of each rating from the user's average, creating centered rating vectors that account for different user rating scales.

These deviations are multiplied together to form a numerator, while the denominator is calculated as the product of the square roots of the summed squared deviations. The final correlation coefficient ranges from -1 to 1, where 1 indicates perfect positive correlation, 0 indicates no correlation, and -1 indicates perfect negative correlation.

Pearson correlation is particularly valuable for recommendation systems because it normalizes each user's ratings around their personal average, effectively addressing the issue of users who consistently rate items higher or lower than others, thus focusing on rating patterns rather than absolute values.

Performance

- **MAE of GIVEN 5:** 0.84631736901338
- **MAE of GIVEN 10:** 0.7766666666666667
- **MAE of GIVEN 20:** 0.755377640590335
- **OVERALL MAE:** 0.790469545230668

User-IUF The Inverse User Frequency (IUF) is the third **user-based** algorithm and extends **user-pearson** by weighting item ratings based on their popularity. It operates on the principle that items rated by fewer users are more informative and should have greater influence in similarity calculations.

IUF applies a logarithmic weighting factor to each item where the weight is inversely proportional to the item's popularity. This is calculated as $\log(\text{total_users} / \text{item_frequency})$. When a movie is rated by fewer users, its IUF value increases, giving it more significance in similarity computations.

The implementation first tallies how many users have rated each item to determine frequency. Then, it calculates the IUF value for each item. During similarity calculations, these weights are incorporated into the Pearson correlation similarity metric, amplifying the contribution of rare items while reducing the influence of commonly-rated items.

This approach is particularly effective in systems where some items are universally popular and thus less effective at distinguishing user preferences. By emphasizing uncommon ratings, IUF can highlight the relationships between users and potentially improve recommendation quality, especially for niche interests.

Performance

- **MAE of GIVEN 5:** 0.821057896711267
- **MAE of GIVEN 10:** 0.759
- **MAE of GIVEN 20:** 0.738497154432333
- **OVERALL MAE:** 0.770645214250534

User-Case Last user-based algorithm I worked on is Case Modification algorithm that also extends **user-pearson** and enhances collaborative filtering by adjusting similarity scores based on the number of co-rated items between users. It addresses a common issue in recommendation systems where high similarity scores based on very few common items may be statistically unreliable.

This approach starts with a standard similarity metric (Pearson correlation in this implementation) but then applies a significance weighting factor. The factor is determined by comparing the number of items two users have both rated against a predefined threshold (50 items in this implementation). When users share fewer than the threshold number of items, the similarity score is penalized proportionally.

The penalty is calculated using a power function: $(\min(n, \text{threshold}) / \text{threshold})^{1.25}$, where n is the number of co-rated items. This creates a dampening effect that becomes more pronounced as the overlap between users decreases. The exponent (1.25) controls how aggressively the algorithm penalizes small overlaps.

By incorporating this adjustment, the Case Modification algorithm gives more weight to similarity scores based on substantial evidence while reducing the influence of potentially coincidental matches. This should help to produce more reliable recommendations by favoring user relationships established through multiple shared preferences rather than limited interactions.

Performance

- **MAE of GIVEN 5:** 0.843566337376516
- **MAE of GIVEN 10:** 0.7718333333333333
- **MAE of GIVEN 20:** 0.743223690556574
- **OVERALL MAE:** 0.783204728287638

Item-Based Collaborative Filtering

I only implemented one **item-based** algorithm that is based on cosine similarity.

Item-Based This **item-based** algorithm shifts the approach by focusing on relationships between items rather than users. My implementation calculates similarities between movies based on how users have rated them, creating a model that can predict a user's preference for unseen movies.

The algorithm computes cosine similarity between movie pairs by examining users who have rated both items. For each movie pair, it calculates the dot product of their rating vectors and normalizes by their magnitudes. To address statistical reliability concerns, the implementation applies a shrinkage factor that penalizes similarities based on few common users, making the system more robust against coincidental correlations.

When predicting ratings, the algorithm uses a weighted average approach incorporating baseline averages. For each movie needing prediction, it examines movies the user has already rated, weights their deviation from average by similarity scores, and applies this weighted deviation to the target movie's baseline average. This effectively transfers a user's opinion patterns from known movies to new ones.

Performance

- **MAE of GIVEN 5:** 0.810679004626735
- **MAE of GIVEN 10:** 0.7398333333333333
- **MAE of GIVEN 20:** 0.721520208353429
- **OVERALL MAE:** 0.755294697094073

Hybrid/Ensemble Collaborative Filtering

These algorithms in one way or another combine **user-based** and **item-based** methods to leverage the strengths of both approaches.

Hybrid This was my first attempt at creating a **hybrid** algorithm that combines **user-based** and **item-based** approaches to create a more robust recommendation system that uses the strengths of both methods. This implementation integrates both approaches with additional statistical techniques to improve prediction accuracy.

At its core, the algorithm initializes with both **user-based** and **item-based** collaborative filtering models, calculating supplementary metrics like movie popularity, user mean ratings, and rating standard deviations. These metrics enable the algorithm to account for different user rating scales and biases through normalization techniques.

When predicting ratings, the hybrid approach generates separate predictions from both the user-based and item-based models, then combines them using a weighted average. The weighting dynamically adjusts based on the user's profile density - as users rate more items, the user-based component receives increased weight (up to 0.8), reflecting growing confidence in user-similarity data. For users with sparse ratings, item-based recommendations receive greater weight.

A distinctive feature of my implementation is its confidence weighting mechanism. Movies with higher popularity receive greater confidence in their predictions through a sigmoid function that maps popularity to a 0.5-1 range. This confidence weight then balances the hybrid prediction against the movie's average rating, effectively tempering predictions for obscure movies toward the movie's consensus rating.

Performance

- **MAE of GIVEN 5:** 0.795923471301738
- **MAE of GIVEN 10:** 0.7315
- **MAE of GIVEN 20:** 0.727886563133018
- **OVERALL MAE:** 0.751108192415039

SVD My second attempt at a **hybrid** algorithm was Matrix Factorization based recommender using Singular Value Decomposition (SVD). It works by decomposing the original user-item rating matrix into lower-dimensional representations, capturing latent features that underlie user preferences and item characteristics.

This implementation uses stochastic gradient descent to iteratively optimize user and item feature vectors. Each user and movie is represented by a feature vector of configurable length (default 50 dimensions), initialized with slight random variations around a scaled average. The algorithm also incorporates bias terms for both users and movies to account for their baseline rating tendencies, addressing the fact that some users tend to rate higher or lower than others, and some movies consistently receive higher or lower ratings.

During training, the algorithm iteratively processes known ratings, calculating the error between predicted and actual ratings and adjusting the feature vectors and biases accordingly. The adaptive learning rate mechanism reduces the learning rate when improvements slow down, preventing oscillation and ensuring convergence. Regularization terms are applied proportionally to the inverse square root of rating counts, helping to prevent overfitting while allowing appropriate flexibility for users or items with more data.

For new users with some known ratings, the algorithm employs a "folding-in" method that learns user features through a mini-optimization process focused just on that user's known ratings. The final prediction system blends pure SVD predictions with baseline estimates, where the blending factor depends on the confidence in the prediction based on user activity and movie popularity. This hybrid approach particularly helps with cold-start scenarios where limited data is available, smoothly transitioning from collaborative filtering to simpler baseline predictions when necessary.

Performance

- **MAE of GIVEN 5:** 0.800175065649619
- **MAE of GIVEN 10:** 0.739166666666667
- **MAE of GIVEN 20:** 0.725282145268641
- **OVERALL MAE:** 0.753283533081596

Hybrid-SVD My last attempt at a **hybrid** algorithm was a combination of the Hybrid and SVD algorithms. This implementation tries to address the common limitations of individual recommendation algorithms by combining their strengths.

The algorithm begins by computing user and movie statistics, including mean ratings, standard deviations, and popularity metrics. These enable personalized prediction adjustments based on individual rating patterns. It also pre-computes neighborhoods for both users and movies to improve recommendation efficiency, focusing on the most active users and popular movies to balance computational resources with coverage.

When making predictions, the system analyzes the user's rating behavior to determine optimal weighting strategies. It examines whether the user is strict or generous in ratings, if they have polarized opinions, and whether they prefer mainstream content. Based on this analysis, the algorithm dynamically adjusts the weights assigned to each component recommender. For users with few ratings, it relies more heavily on **item-based** and direct movie similarity approaches. For users with mainstream tastes, SVD receives greater emphasis, while polarized users benefit more from **user-based** recommendations.

It's also using the confidence-weighted blending mechanism that incorporates movie popularity. Predictions for popular movies receive higher confidence and rely more on the ensemble prediction, while obscure movies are tempered toward their average ratings. The system also applies a bias correction reflecting the user's general rating tendency compared to the global average.

The final prediction undergoes an adjustment phase that modifies ratings based on the user's behavioral pattern. Strict users have their high predictions tempered downward, generous users have their low predictions adjusted upward, and polarized users have their predictions pushed further from the midpoint. This sophisticated calibration ensures recommendations align with each user's

individual rating style, delivering a highly personalized experience that balances algorithmic intelligence with human preference patterns.

Performance

- **MAE of GIVEN 5:** 0.794172814805552
- **MAE of GIVEN 10:** 0.730333333333333
- **MAE of GIVEN 20:** 0.724124626217806
- **OVERALL MAE:** 0.748645542603842

Analytics and Comparison

The performance of each algorithm was evaluated using the Mean Absolute Error (MAE) metric on the test datasets. The MAE measures the average absolute difference between predicted and actual ratings, providing a quantitative assessment of recommendation accuracy.

The results show that the **hybrid** and **hybrid-svd** algorithms consistently outperformed other approaches across all test datasets. These hybrid models leverage the strengths of multiple recommendation techniques, combining **user-based**, **item-based**, and statistical methods to produce more accurate predictions. By dynamically adjusting weights based on user behavior and movie popularity, these algorithms deliver personalized recommendations that align closely with user preferences.

The **SVD** algorithm also performed well, demonstrating the effectiveness of matrix factorization techniques in capturing latent features underlying user-item interactions. By decomposing the rating matrix into lower-dimensional representations, SVD can identify complex patterns and relationships that drive user preferences, leading to accurate predictions even with sparse data.

In contrast, the **user-based** and **item-based** algorithms showed slightly lower performance, with **user-iuf** and **item-based** performing better than **user-cosine**, **user-pearson**, and **user-case**. These results suggest that while collaborative filtering methods are effective for recommendation systems, hybrid and matrix factorization approaches can offer superior performance by combining multiple strategies and leveraging advanced statistical techniques.

Potential Improvements

To further enhance recommendation accuracy, several improvements could be considered (given more data is provided):

- **Feature Engineering:** Incorporating additional user and movie features, such as genre, release year, or director, could provide more context for recommendations and improve prediction quality.

- **Temporal Dynamics:** Accounting for temporal trends in user preferences and movie popularity could help adapt recommendations to changing tastes and ensure relevance over time.
- **Deep Learning:** Exploring deep learning models, such as neural collaborative filtering or recurrent neural networks, could capture complex user-item interactions and improve prediction accuracy.